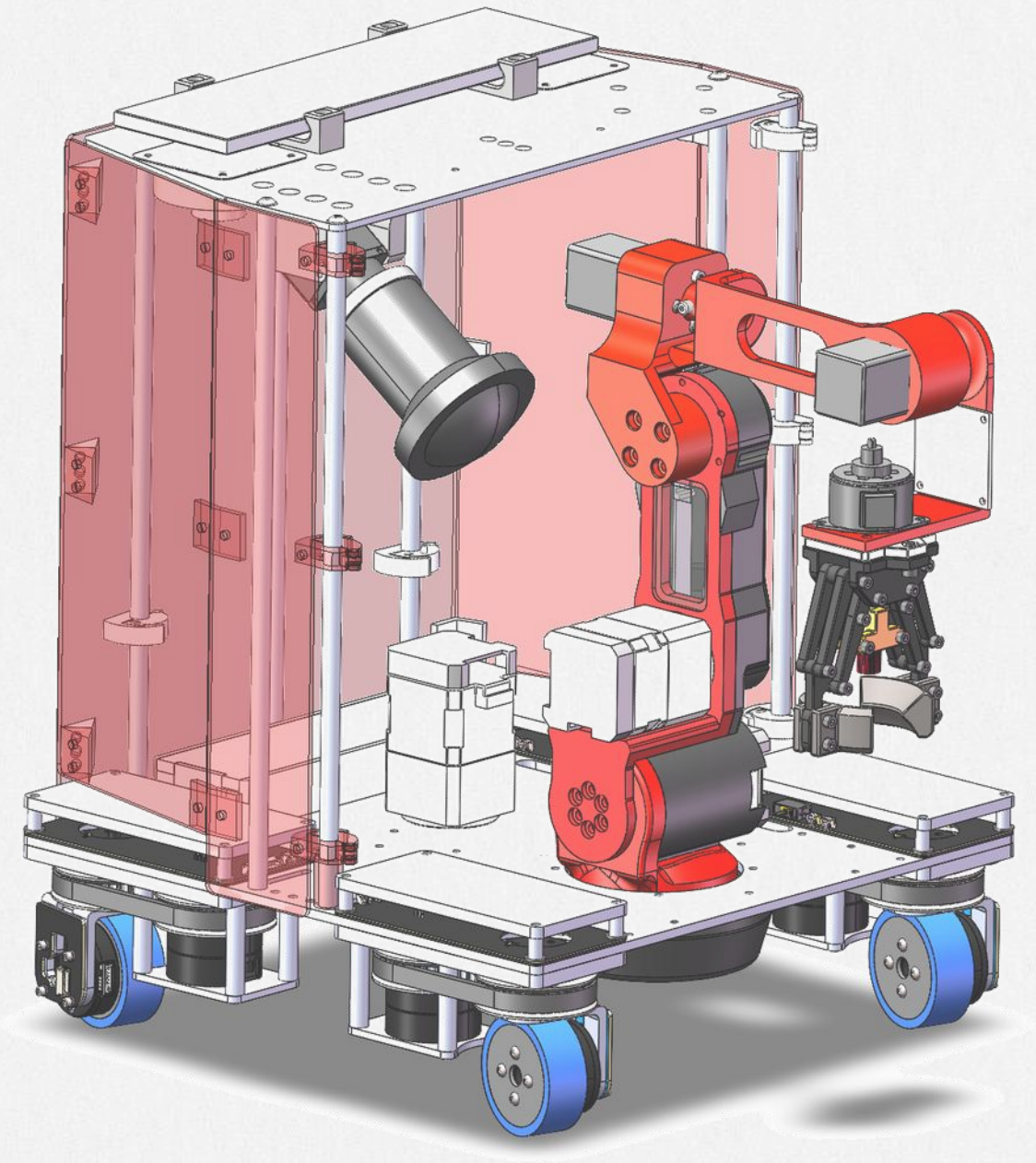


FINS Lab

Future Intelligent Network System Lab

Fines Robot: Hardware, Algorithms, and Software Ecosystem

Connecting Intelligence, Empowering the Future

A Question

What determines how well a robot performs

Competition robot

Off-the-shelf robot

They use the same technology,

so what sets them apart?

Different actuator capability

but that is not the core difference

Looks more agile

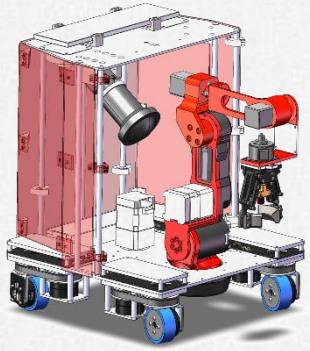
every algorithm layer runs at a higher rate,
with better real-time behavior

Runs more functions at once

the program logic is not a single linear flow

Anatomy of a Robot Solution

A general robot solution



Robot body

Controller & circuits

FineMote

FineVision

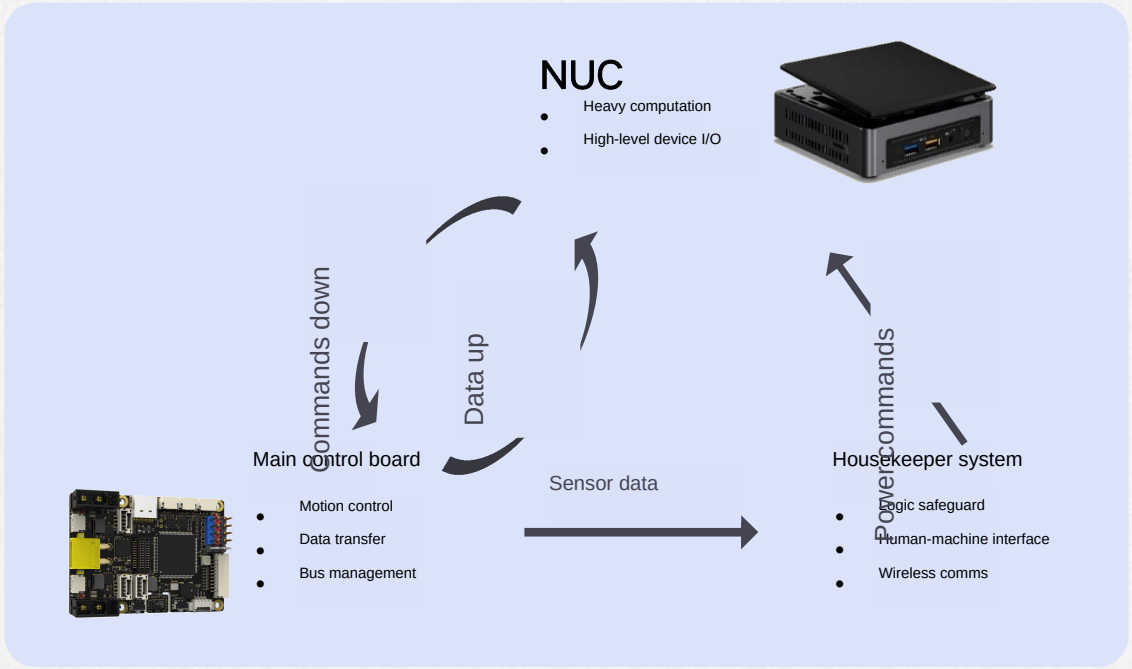
.....

Software ecosystem

Platform

System Architecture

Hardware Structure



Brain High-level intelligence

Cerebellum Brainstem

Motion regulation Vital center

The three core devices mirror the structure of the human brain by analogy.

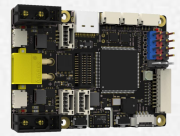
Hardware Structure

Host and embedded controller each play their part

Host: onboard computer

- Supplies compute for heavy workloads (it is a PC after all)
- Hosts drivers for cameras, LiDAR and similar devices (Ubuntu)

Embedded: main control board



- Real-time control (MCU territory)
- Sensor data aggregation

Housekeeper system

- Closed to users; maintains base robot functions
- Provides health monitoring and the human interface

Traits

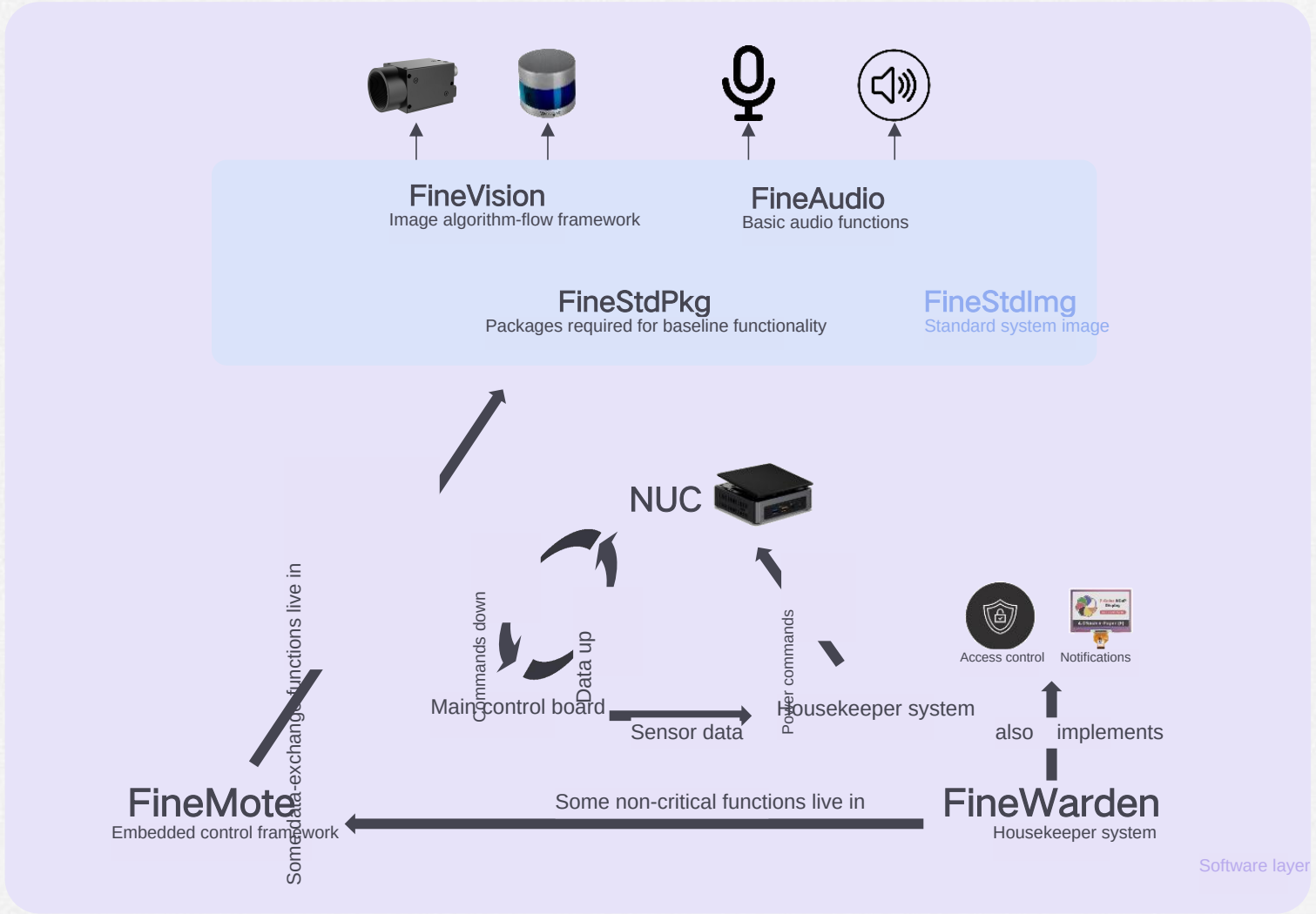
- Three devices jointly deliver robot control

Benefits

- Balances compute demand against real-time behavior
- The housekeeper keeps the robot safe when users make mistakes

System Architecture

Software ecosystem



Software ecosystem

Mapped onto the hardware structure

Main board -> embedded framework

FineMote

NUC -> host software ecosystem

FineVision vision algorithm framework

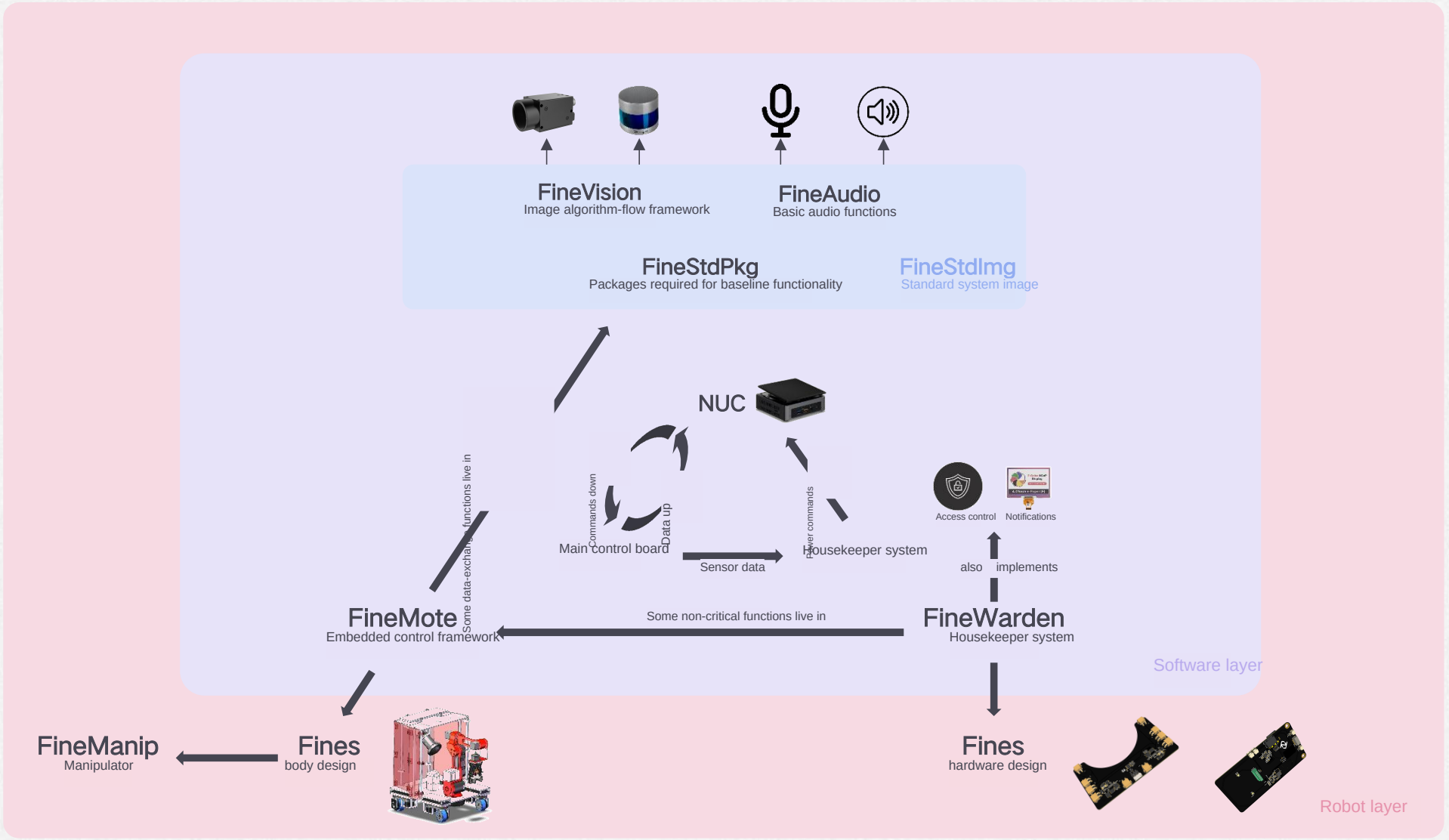
FineStdImg standard system image

Housekeeper -> housekeeper firmware

FineWarden

System Architecture

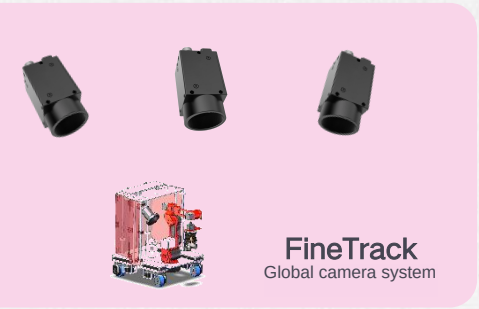
Project Framework

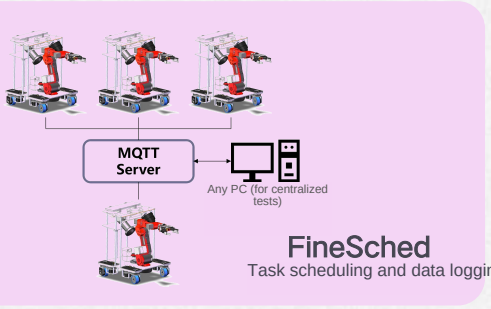


Project Architecture

Developed in stages across layers

Building on the hardware structure, Fines splits into several sub-modules

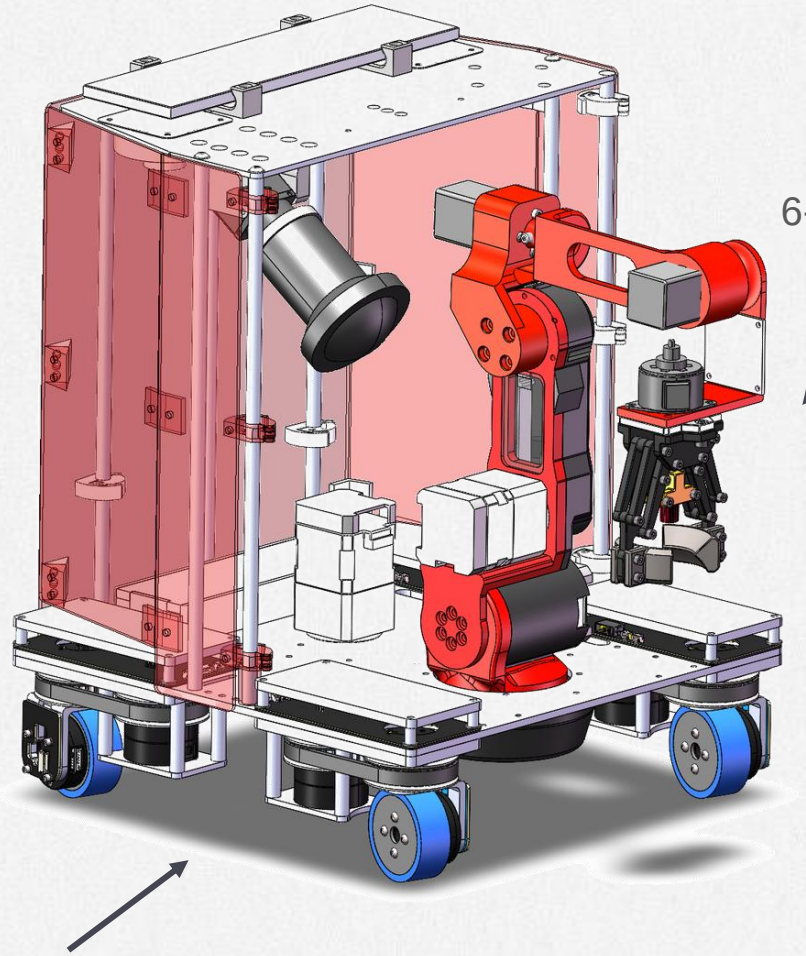




The Fines Robot Family

The best experimental mobile manipulator in the country (to be done)

Fines robots, their hardware, algorithms and software ecosystem



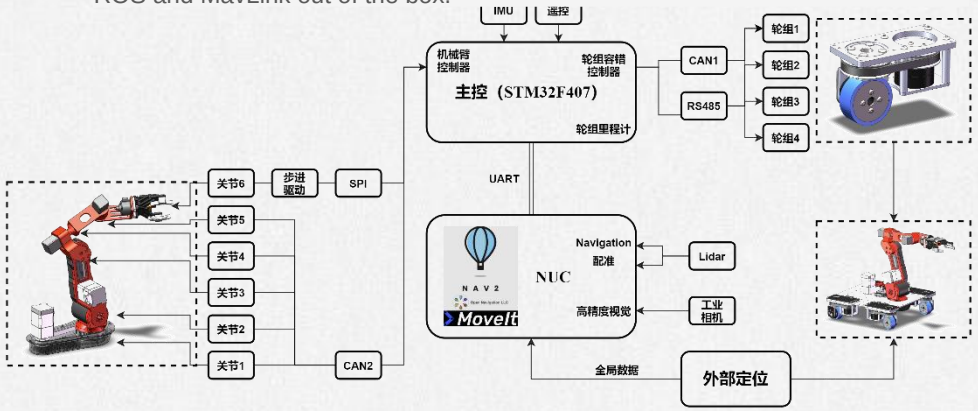
6-DoF manipulator

Steerable-wheel base (POV)

Pseudo-Omnidirectional Vehicle

A robotics research platform built for what comes next

Fines breaks with the usual research platform, the kind that is crude, fragile and far from real production. The design was rebuilt across the whole manufacturing chain to deliver smooth high-precision control with better reliability. It speaks industrial fieldbus, so a production line can be simulated, and it supports common middleware such as ROS and MavLink out of the box.



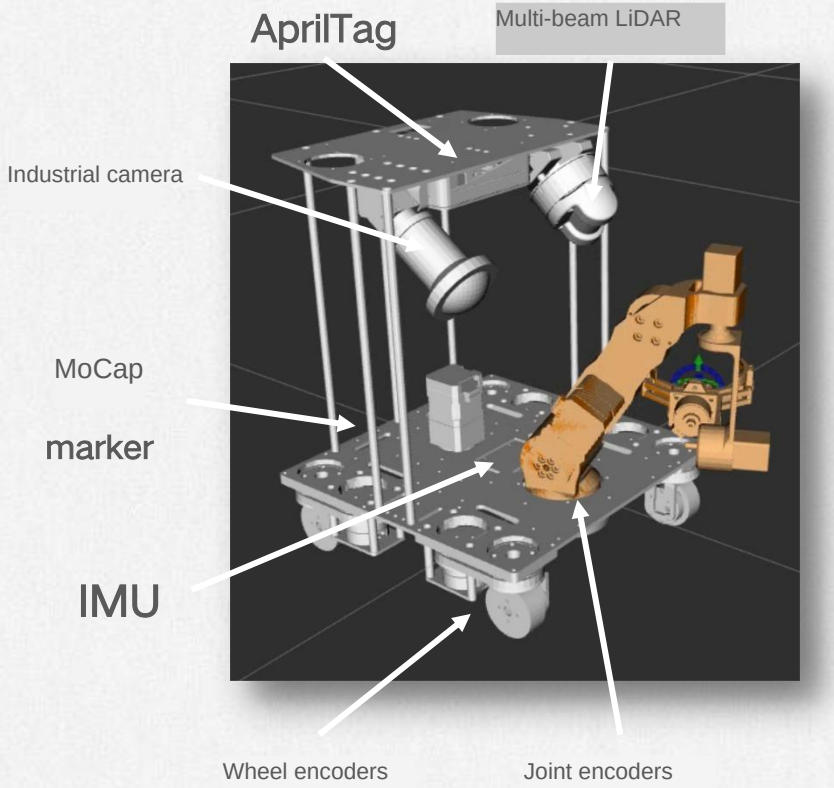
Size: 300 * 300 * 400 mm
Accuracy: 1% open-loop, 0.3 mm global closed-loop, 0.05 mm manipulator
Max drive force: 23.07 N
Manipulator payload: 10 N

Sensing System

Part One

Sensing System

A systematic sensing design



Three core questions

What can a camera give us?

Industrial camera, spatial vision

What can a point cloud give us?

Multi-beam LiDAR, point cloud / mesh map

How do we calibrate?

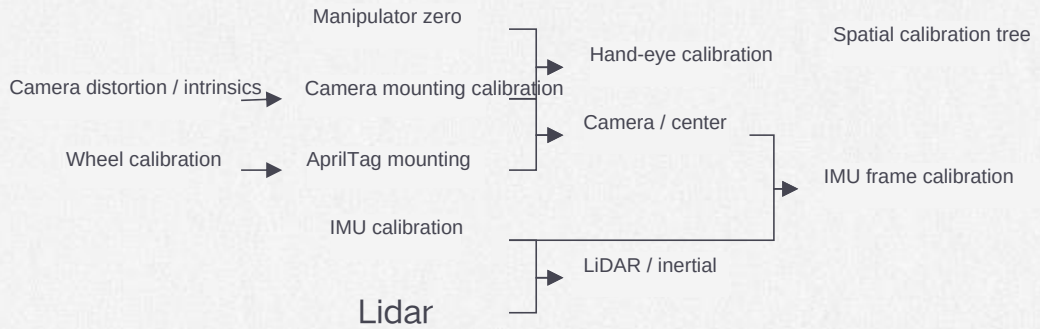
Temporal and spatial calibration

Temporal calibration

Synchronize sensor data

Spatial calibration

Align every sensor frame



Sensing System

Integrating sensor data systematically

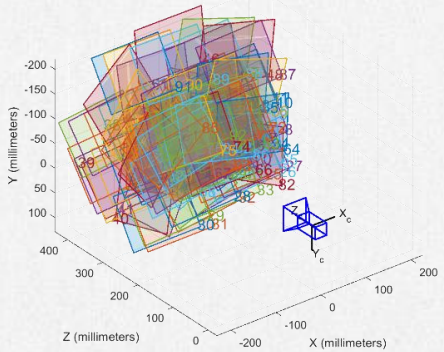
Calibration: Camera

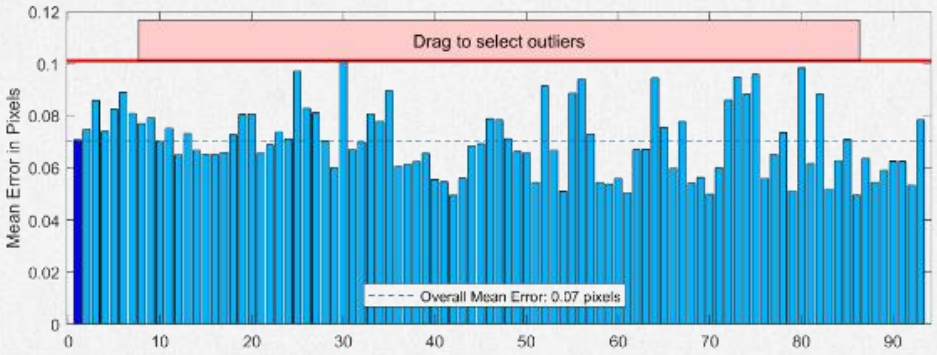
The key to sensing the environment accurately

Intrinsics and distortion calibration

Uniform sampling across the field of view

Outliers are filtered out to push calibration accuracy further

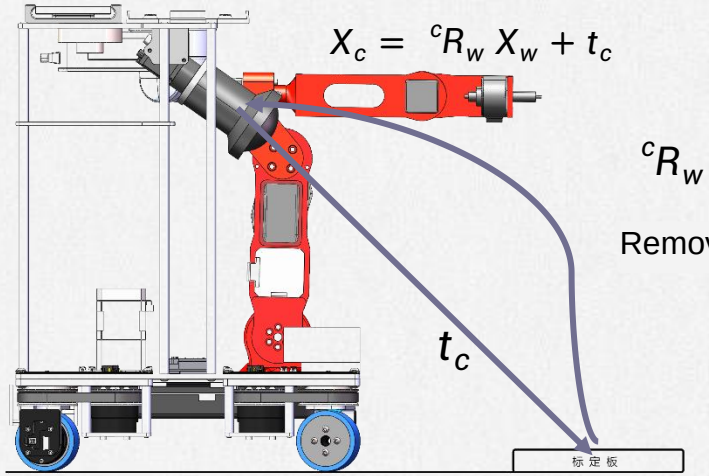




Camera mounting calibration

Perspective–n–Point

The imaging plane cannot be measured directly, so a calibration tool recovers the two rotational degrees of freedom of the 3x3 rotation matrix, together with the vertical component of the translation from the optical center to the calibration plane.

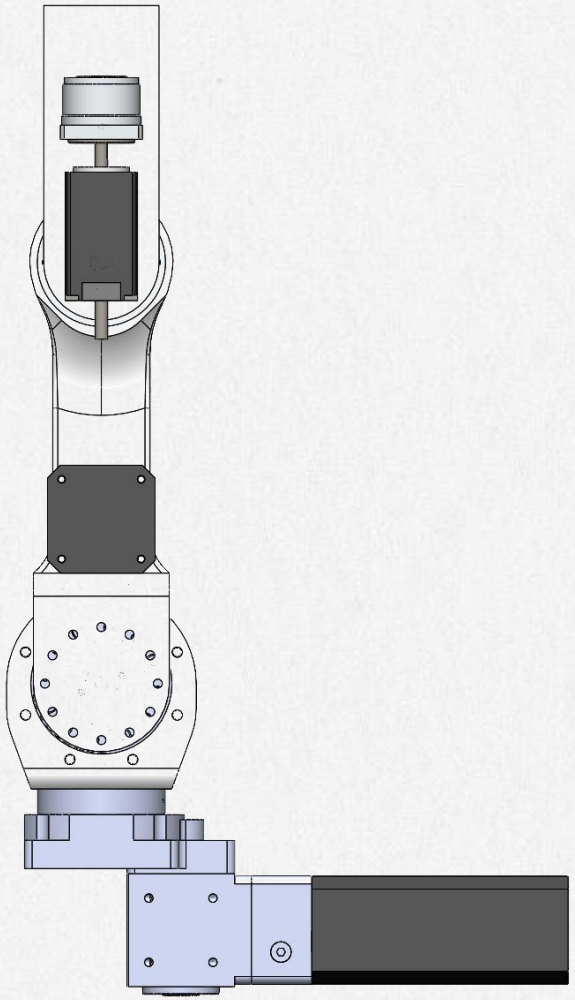


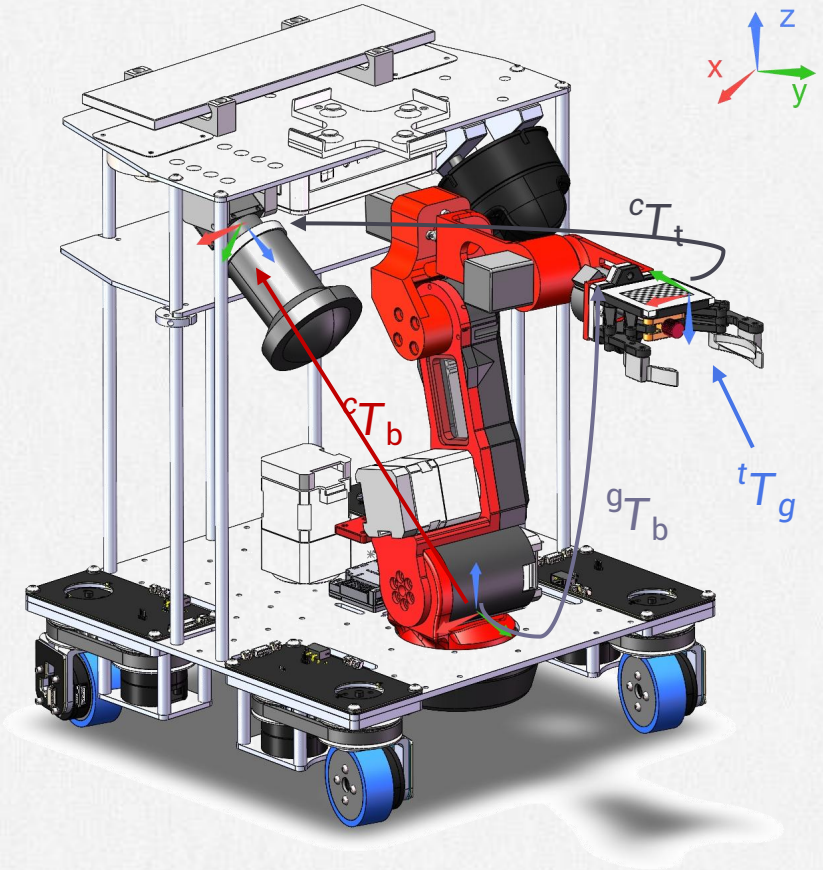
$${}^cR_w = {}^c_1R_w * {}^c_2R_w * {}^c_3R_w$$

Removes misaligned-board error R_w

Calibration: Manipulator

FineManip





Zero-position calibration

A fixture holds the arm fully extended upward at its highest pose, then every joint encoder is zeroed.

Manipulator calibration

The last step toward tight coordination

Hand-eye calibration (eye-to-hand)

The camera frame origin and basis cannot be measured externally, so hand-eye calibration recovers the camera-to-arm frame relationship.

- the homogeneous transform to be calibrated
- obtained from forward kinematics
- unknown, and not needed
- obtained through PnP

$$({}^{g}_{2}T_{b})^{-1} \cdot {}^{g}_{1}T_{b} \cdot {}^{b}T_{c} = {}^{b}T_{c} \cdot {}^{c}_{2}T_{t} \cdot ({}^{c}_{1}T_{t})^{-1}$$

$$A \cdot X = X \cdot B$$

Solving X over many samples yields the camera-to-arm relationship

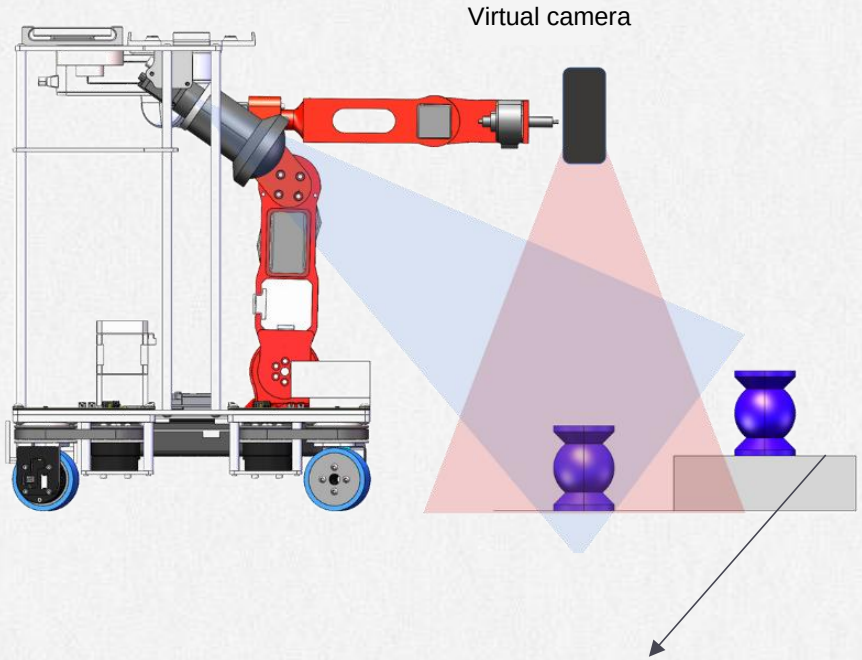
Camera: Spatial Visual Localization

The key to sensing the environment accurately

Intrinsics -> mounting -> homo-

graphy

Spatial vision in three steps



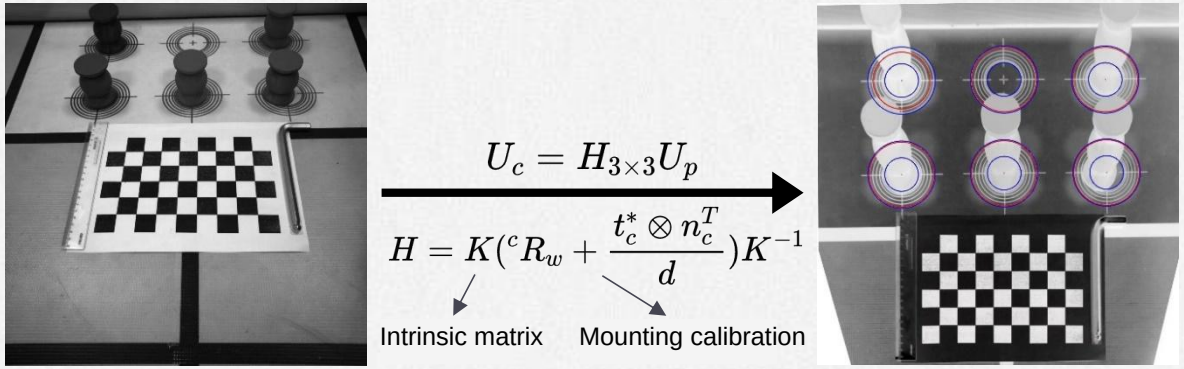
The virtual top-down view makes features easier to extract and recognize

With the Z height known, the spatial position of each feature can be computed

Bird's-eye view

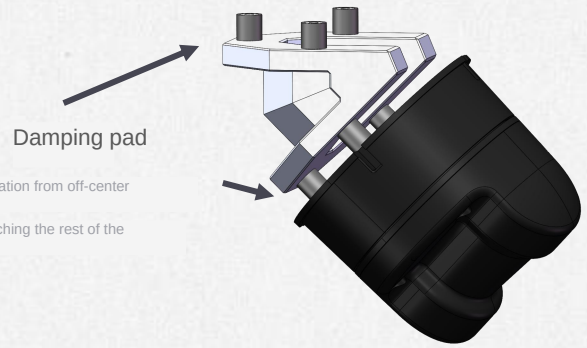
Homography

For a plane at a known Z height, a homography produces the image a virtual camera would see looking straight down, which makes features far easier to recognize.

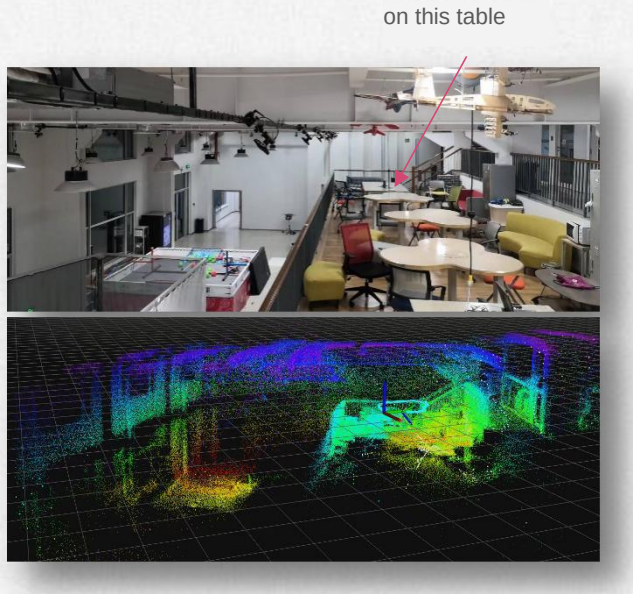


Point cloud: LiDAR-Inertial Odometry

Point-cloud Mapping



LiDAR mounting

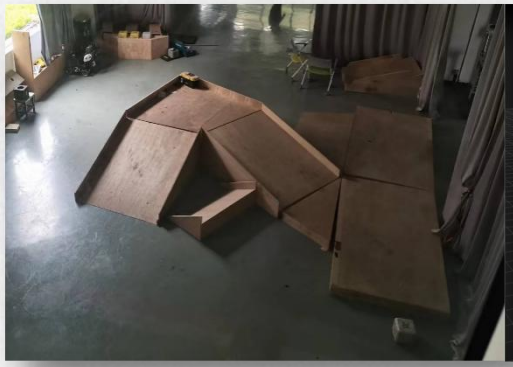


Example: static scan of a large scene

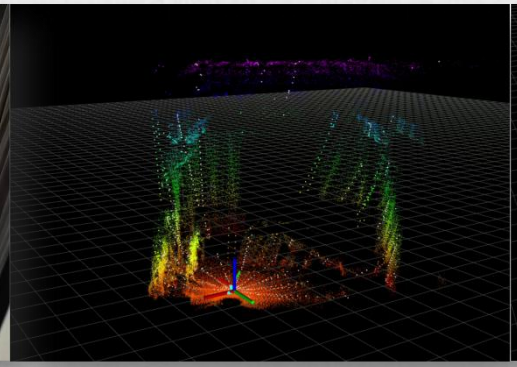
LiDAR-inertial odometry

Builds a reliable point-cloud map incrementally

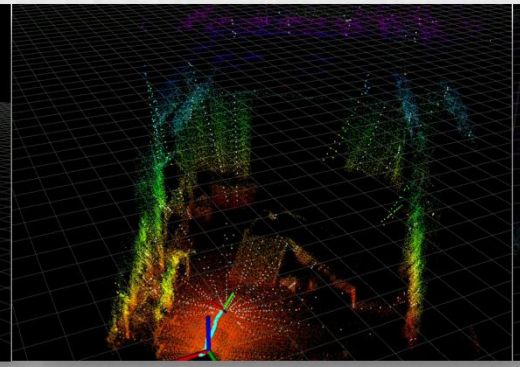
- LiDAR: Unitree 4D LiDAR, 21600 points
- FOV: 360° x 90°
- Odometry: Point-LIO
- Map published as PointCloud2



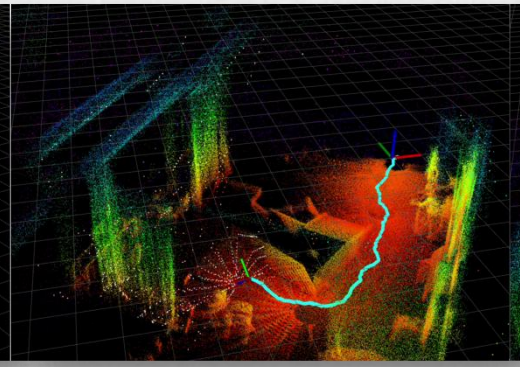
Scene to be mapped



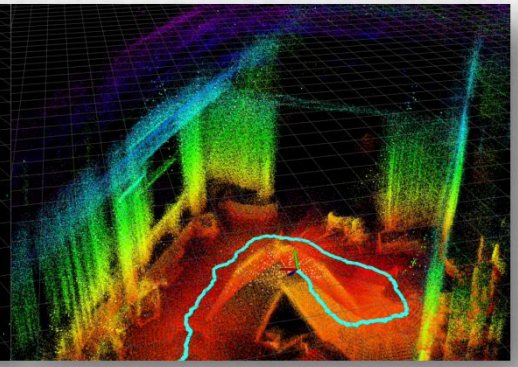
First frame captured, mapping starts



Part of the scene is scanned



Moving behind the scene fills in more features



Climbing to the top completes the map

Example: dynamic scan of a complex scene

Software ecosystem

Part Two

A worked example

Why the Fines software ecosystem

Problem 0: development model

Developers do not know each other's implementation details

Problem 2: thread scheduling

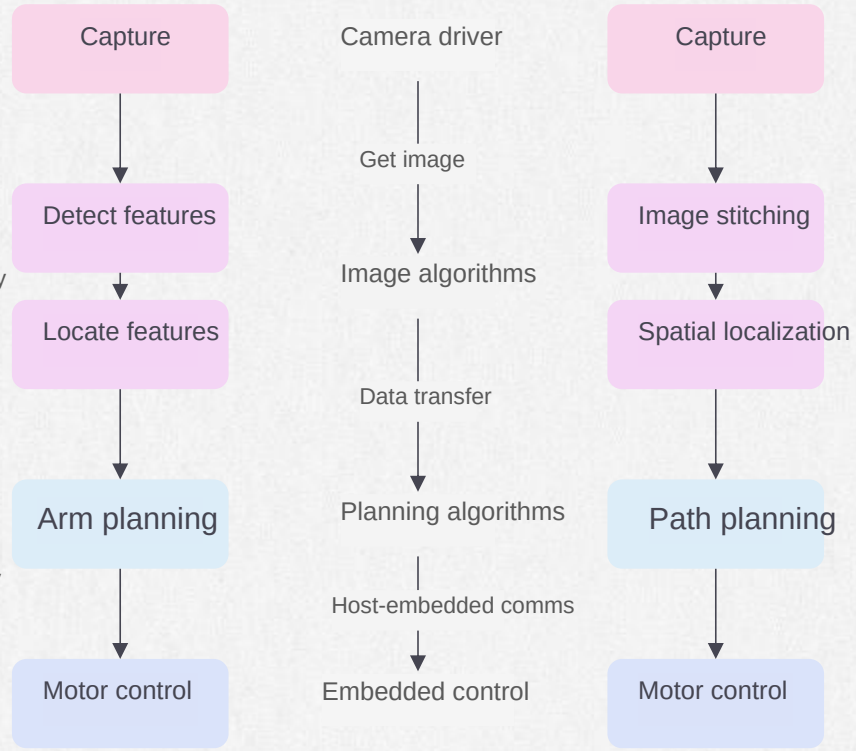
The pipeline has many stages; at high frame rates they must run in parallel

Problem 4: embedded specifics

Embedded work has a steep learning curve and many scattered concerns; the code needs a workable structure

Manipulator application

Base application



Problem 1: device resource conflicts

One camera frame may be used by several user programs at once

Problem 3: dependencies

Each stage depends on the output of the one before it

Problem 5: debugging needs

During debugging the pipeline is rearranged constantly

Pipeline rate is what decides how well the robot performs

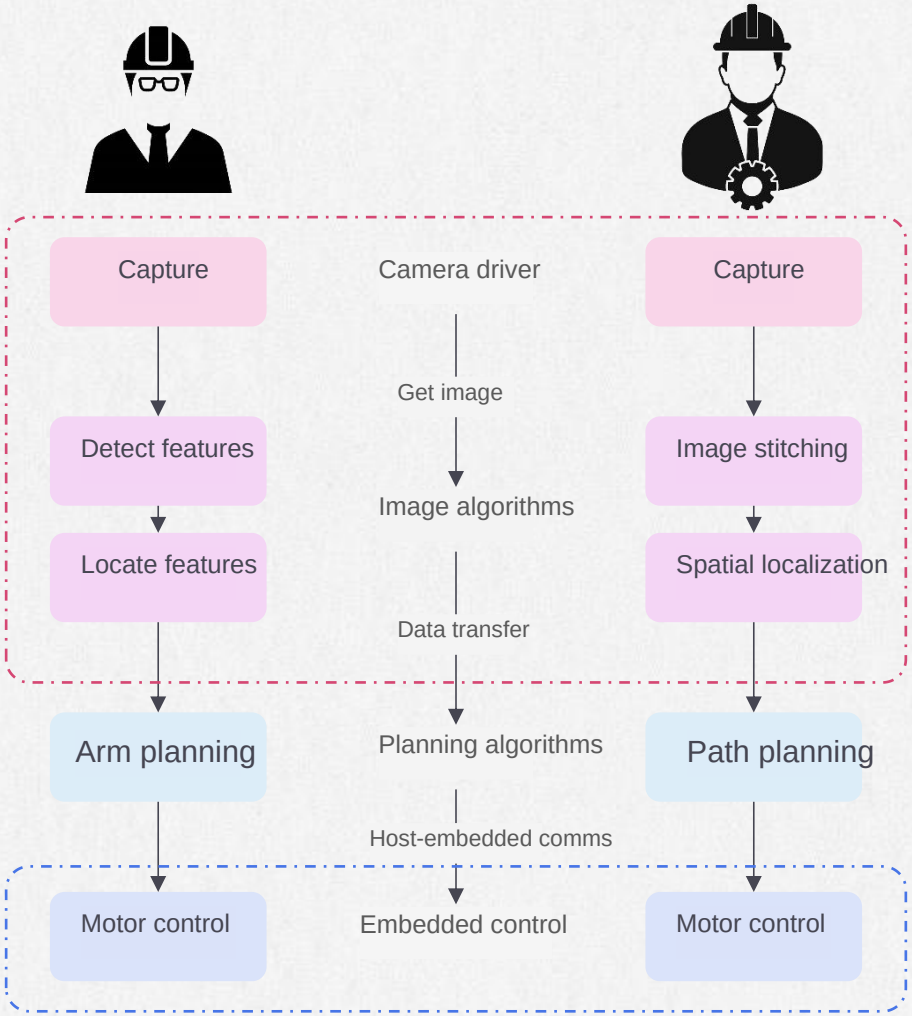
$e^{0.1} - 1 = 10.5\%$ $e^{0.01} - 1 = 1.01\%$ $e^{0.001} - 1 = 0.1\%$

A worked example

Why the Fines software ecosystem

FineVision

- Camera driver
- Streaming image processing
- Visualization and monitoring



FineMote

- Motor & sensor drivers
- Real-time control
- Host-embedded comms

FineVision

Feature Walkthrough

Feature 1: multi-core execution

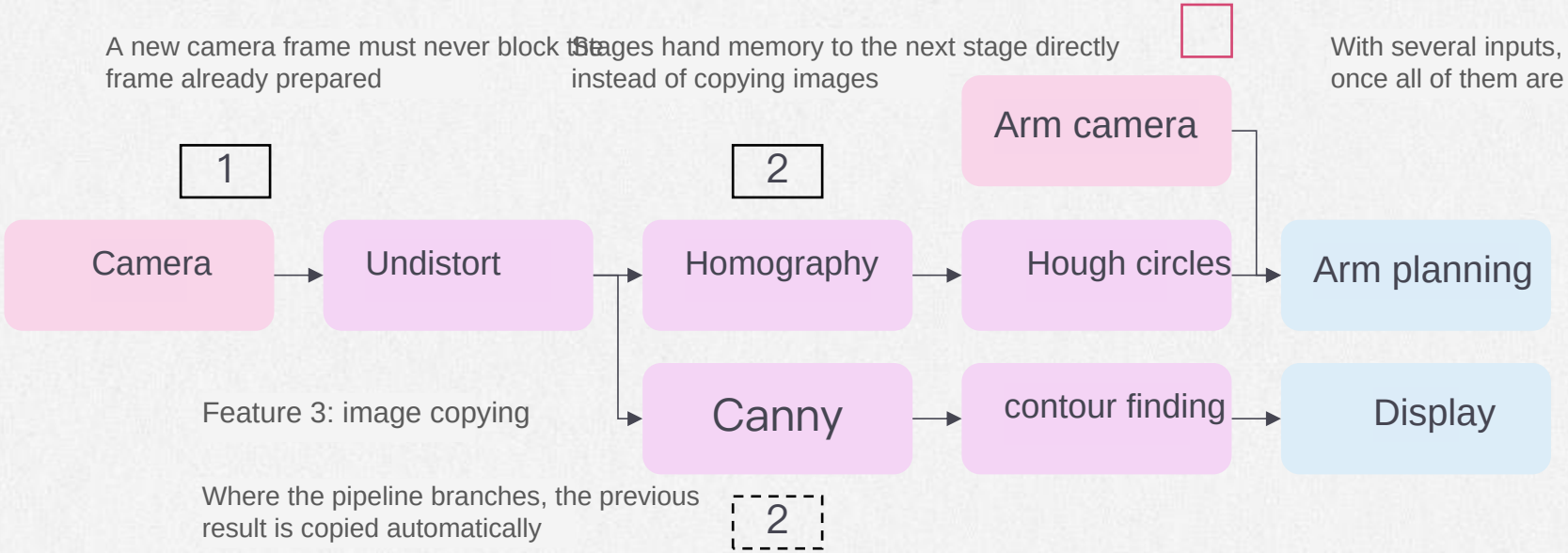
A new camera frame must never block the frame already prepared

Feature 2: image passing

Stages hand memory to the next stage directly instead of copying images

Feature 4: multi-input sync

With several inputs, the next stage wakes only once all of them are ready



Feature 3: image copying

Where the pipeline branches, the previous result is copied automatically

Image source

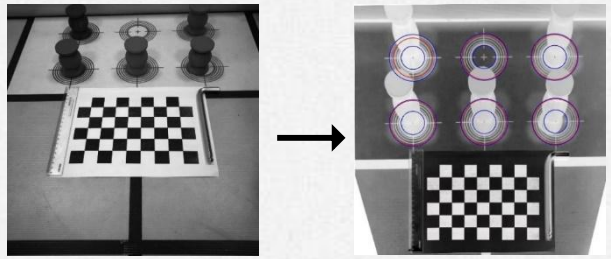
Algorithm node

Stream sink

Image

What users get

- Think in terms of the algorithm flow, not threads
- Mixed C++ / Python with shared data

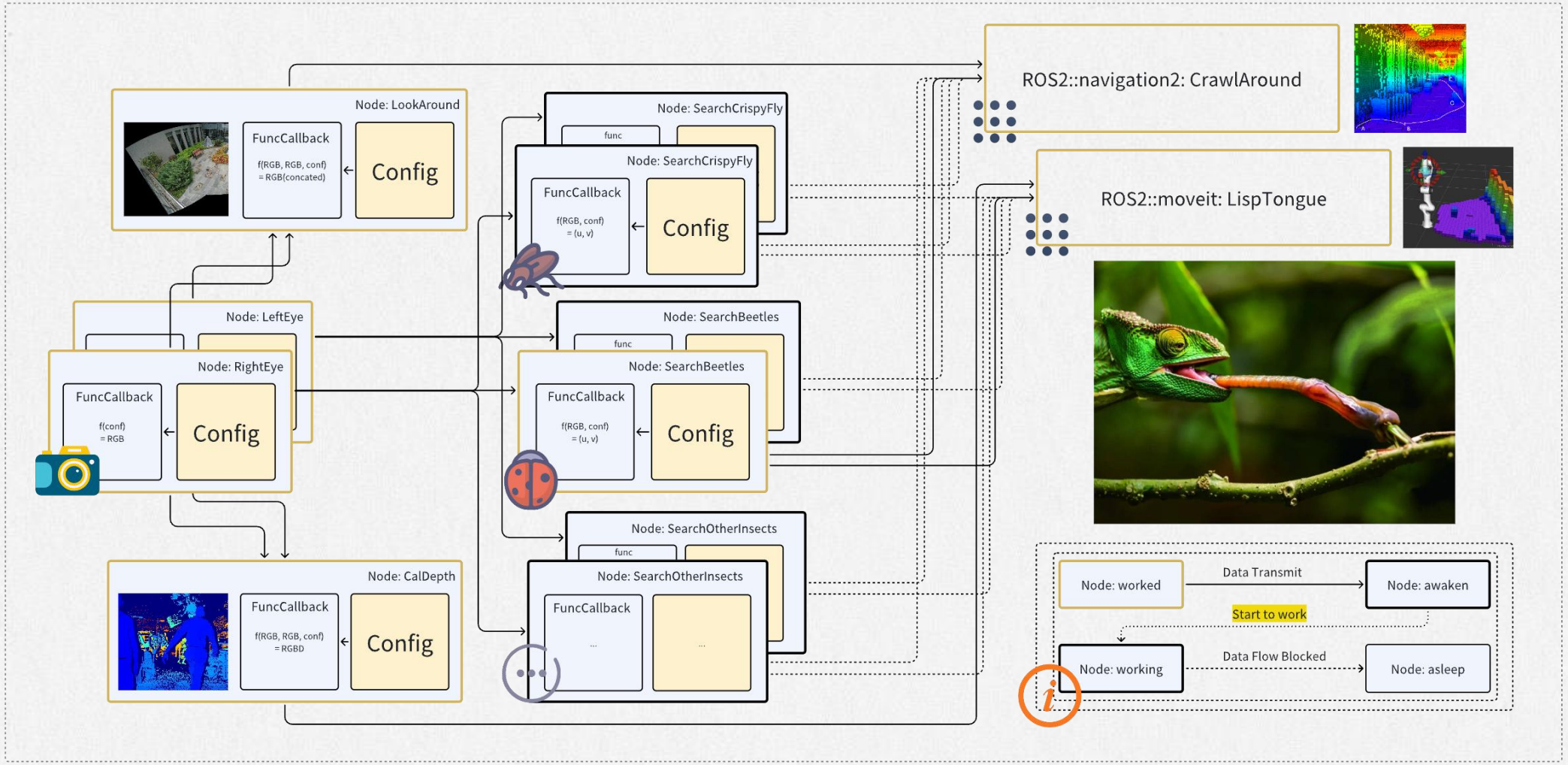


FineVision

A More Complex Example

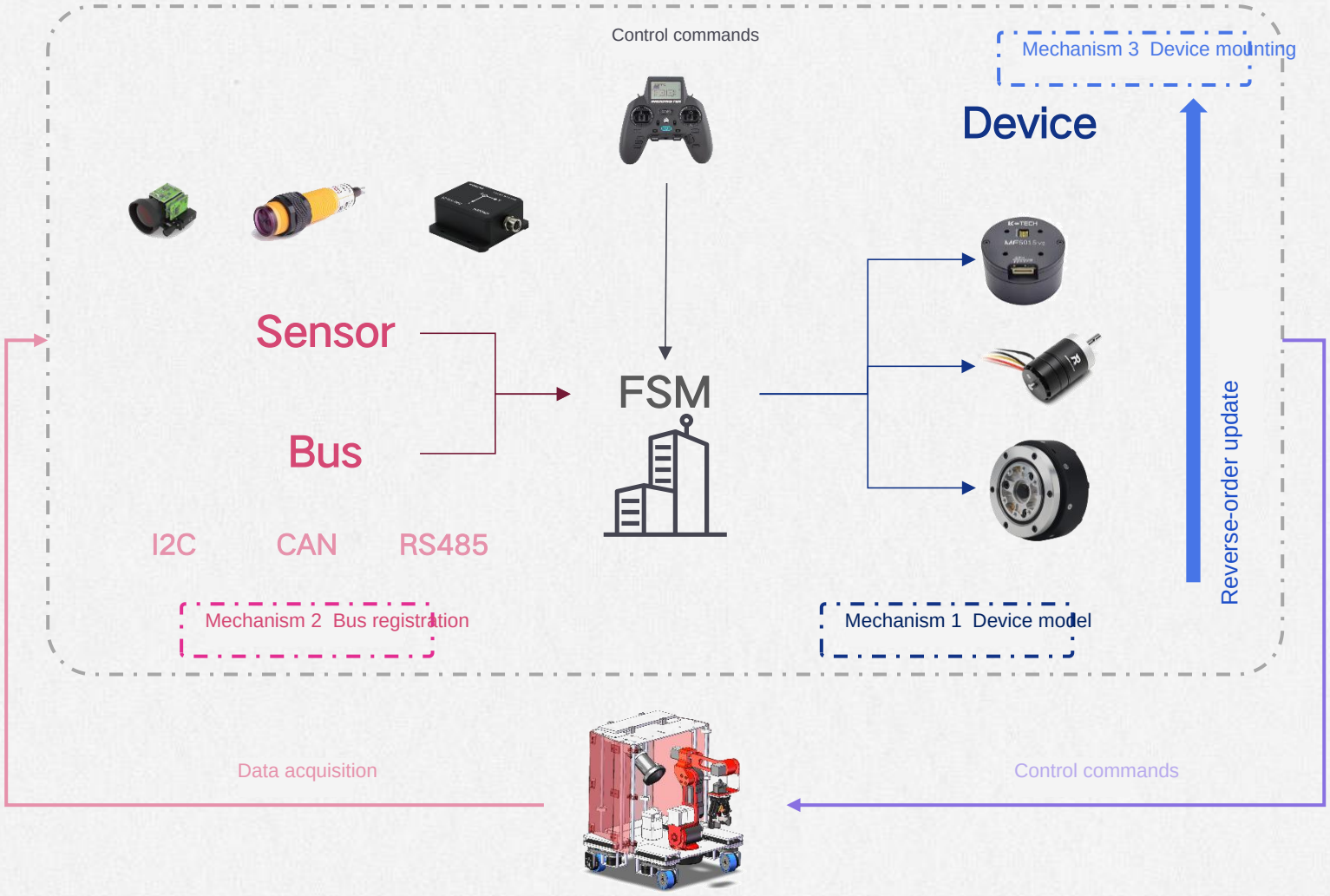
Flow: Predators of a Chameleon

when a beetle comes by



FineMote

A reliable, flexible embedded framework for robots



FineMote

A reliable, flexible embedded framework for robots

Fines splits control between host and embedded board; the embedded side handles sensor data and real-time control.

Broadly applicable

Function

- Runs on many different boards
- Industrial buses supported (CAN, RS485, etc.)
- Works with most off-the-shelf actuators and sensors

How the framework is used

- The framework calls into user code, not the other way around
- Users touch only a handful of files
- No embedded experience required

Application

Modern workflow

Development

- CMake-based project with integrated IDE
- Valgrind analysis and unit tests (on QEMU)

FineMote

A reliable, flexible embedded framework for robots

Pixhawk

STM32F427

Cortex-M4@168MHz

2M Flash 256K RAM

Co-processor

STM32F100

Cortex-M3@24MHz

CUAV V6X

STM32H753

Cortex-M7@480MHz

2M Flash 1M RAM

Co-processor

STM32F10X

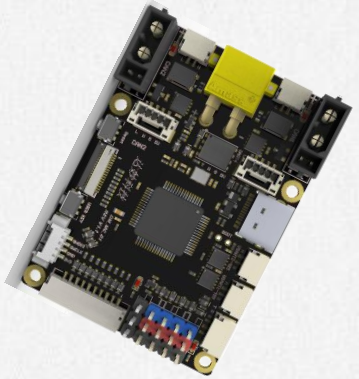
Cortex-M3@72MHz

Mcontroller

STM32H743

Cortex-M7@480MHz

2M Flash 1M RAM



FineMote(DM-MC01)

STM32F446

Cortex-M4@168MHz

512K Flash 128K RAM

Hardware resources are tight

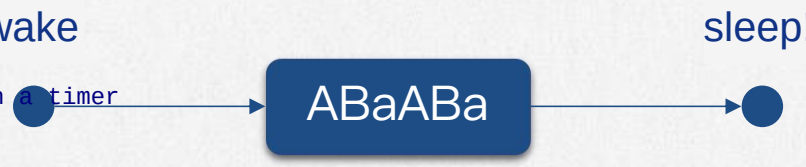
Low clock, little memory, patchy C++ support

and it all has to be cross-compiled

Real-time behavior and raw compute pull against each other; with little RAM, new becomes a luxury, so the code has to stay largely static.

```
class DeviceBase {
private:
        tic std::list<DeviceBase*> deviceList;      // device list of base-class pointers

            id Handle() = 0;                        // pure virtual: derived class overrides it, called on a timer
public:
            void DevicesHandle() {                  // one place that drives every device's periodic execution

   for (auto rit = deviceList.rbegin(); rit != deviceList.rend(); ++rit) {
                  ->Handle();                       // walk the device list in reverse and call its Handle

   };
    }


      deviceList.push_back(this);
    };
    ~DeviceBase();
};
```

Level 0: framework development



Device model

Driven periodically by a timer

Every device runs its own routine on a fixed 1 ms cycle, and the data it needs arrives in the right place within that interval.

```
template<int busID>
class Motor_4010 : public DeviceBase{
                nt<busID> canAgent;                 // CAN bus agent

    void Handle() override {
        uint8_t data = canAgent.rxbuf[0];
    };
public:
    explicit Motor_4010(uint32_t
addr):canAgent(addr){};
    ~Motor_4010(){};
};
```

Level 1: device development

Level 2: user development

```
Motor_4010<1>  motor1(MOTOR_INIT_t{0x141, &PID1,
POSITION});

void BalabalaFunc(){
    motor1.setTargetAngle(90);
}
```

FineMote

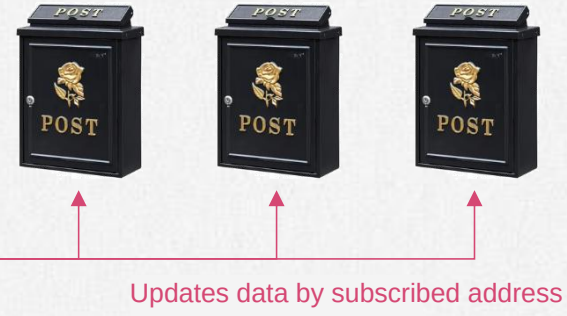
Mechanisms in Detail

```
class DeviceBase {
private:
        tic std::list<DeviceBase*> deviceList;      // device list of base-class pointers

            id Handle() = 0;                        // pure virtual: derived class overrides it, called on a timer

public:
            void DevicesHandle() {                  // one place that drives every device's periodic execution

   for (auto rit = deviceList.rbegin(); rit != deviceList.rend(); ++rit) {
                ->Handle();                         // walk the device list in reverse and call its Handle

   };
    }


      deviceList.push_back(this);
    };
    ~DeviceBase();
};
```

Level 0: framework development

Level 2: user development

```
Motor_4010<1>  motor1(MOTOR_INIT_t{0x141, &PID1,
POSITION});

void BalabalaFunc(){
    motor1.setTargetAngle(90);
}
```



Bus registration

Solves asynchrony between data sources

Devices share bus peripherals, so each one registers with the bus object and then receives exactly the traffic it asked for.

```
template<int busID>
class Motor_4010 : public DeviceBase{
                nt<busID> canAgent;                 // CAN bus agent

 void Handle() override {
        uint8_t data = canAgent.rxbuf[0];
    };
public:
    explicit Motor_4010(uint32_t
addr):canAgent(addr){};
    ~Motor_4010(){};
};
```

Level 1: device development

FineMote

Mechanisms in Detail

```
class DeviceBase {
private:
        tic std::list<DeviceBase*> deviceList;      // device list of base-class pointers

            id Handle() = 0;                        // pure virtual: derived class overrides it, called on a timer

public:
            void DevicesHandle() {                  // one place that drives every device's periodic execution

  for (auto rit = deviceList.rbegin(); rit != deviceList.rend(); ++rit) {
                –>Handle();                         // walk the device list in reverse and call its Handle

   };
    }
```

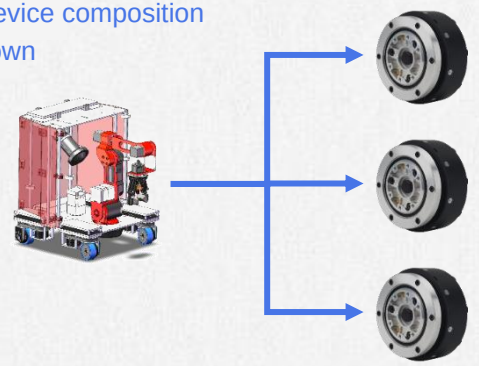
Level 0: framework development

```
      deviceList.push_back(this);
    };
    ~DeviceBase();
};
```

Level 2: user development

```
Motor_4010<1>  motor1(MOTOR_INIT_t{0x141, &PID1,
POSITION});

void BalabalaFunc(){
    motor1.setTargetAngle(90);
}
```

State follows the device composition updated top-down



Device mounting

Init and handlers are called automatically

State updates flow top-down along the device hierarchy, and user changes stay confined to a single file.

```
template<int busID>
class Motor_4010 : public DeviceBase{
                nt<busID> canAgent;                 // CAN bus agent

  void Handle() override {
        uint8_t data = canAgent.rxbuf[0];
    };
public:
    explicit Motor_4010(uint32_t
addr):canAgent(addr){};
    ~Motor_4010(){};
};
```

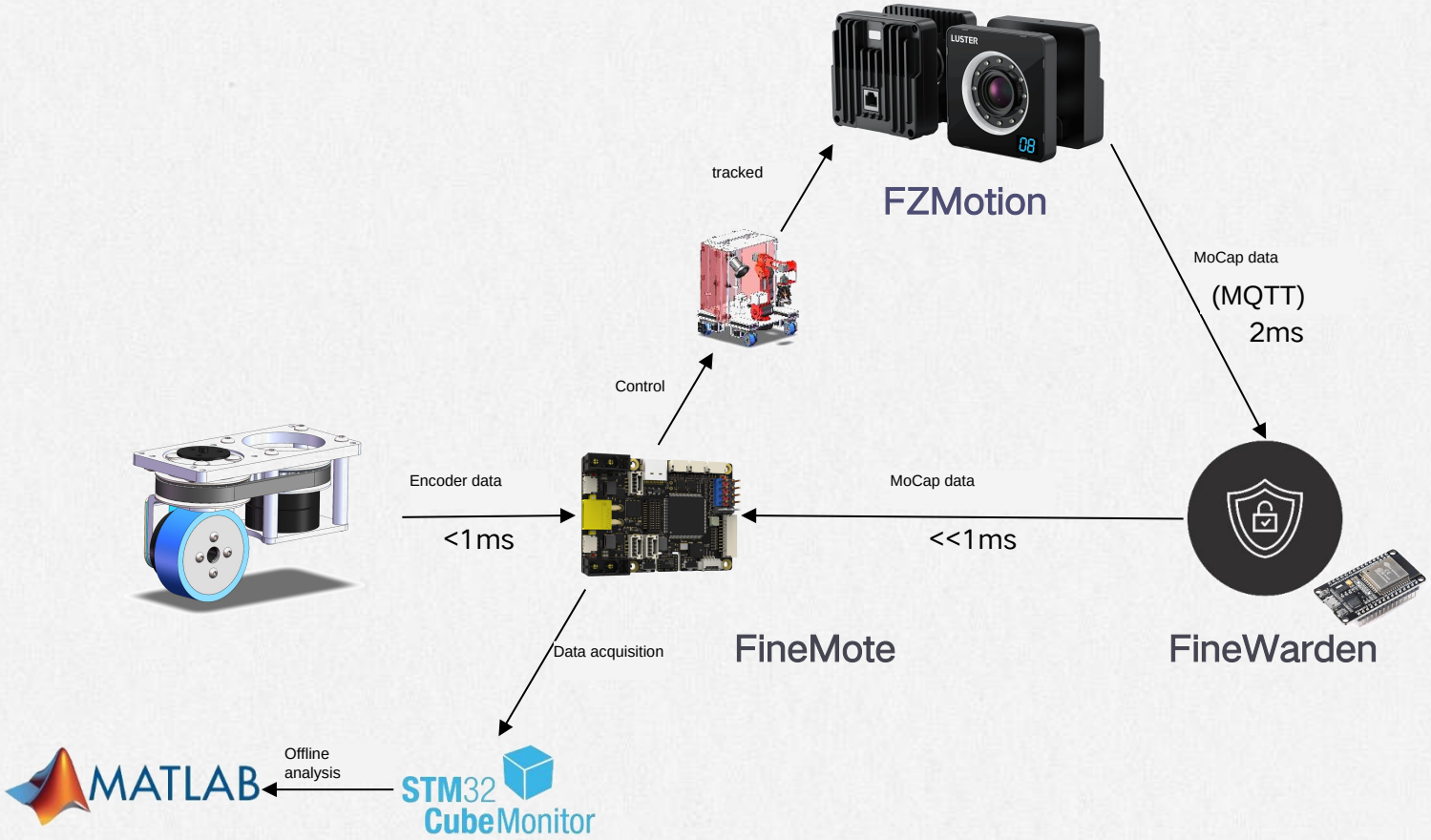
Level 1: device development

Dynamics Optimization

Part Three

Dynamics Optimization

Experiment design

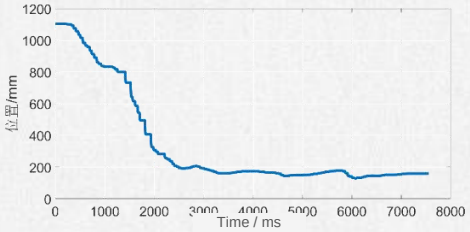


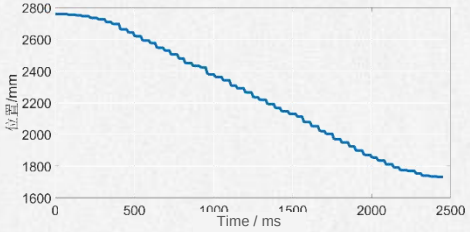
Experiment design

Testing how accurate the model is

Offline analysis of MoCap trajectory data is used
to evaluate how well the model holds up.

The ideal outcome is to regress the per-wheel support-force weights through self-learning. If it converges, the model is accurate.





System Integration

Part Four

FineManip

High-performance, high-reliability manipulator design

Manipulator Design

and parameter validation

- Meets the Pieper condition, so the IK has a closed form

- Sampling the whole workspace to size joint torque demand

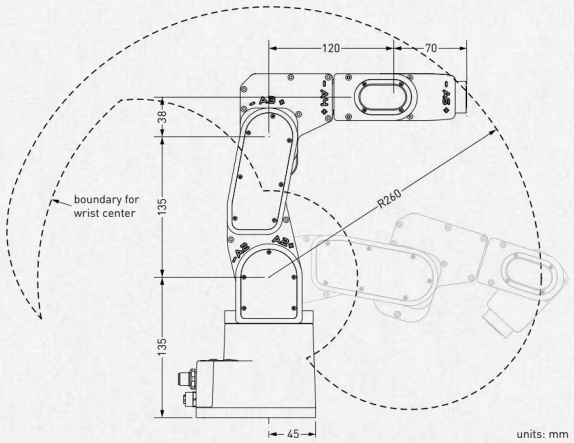
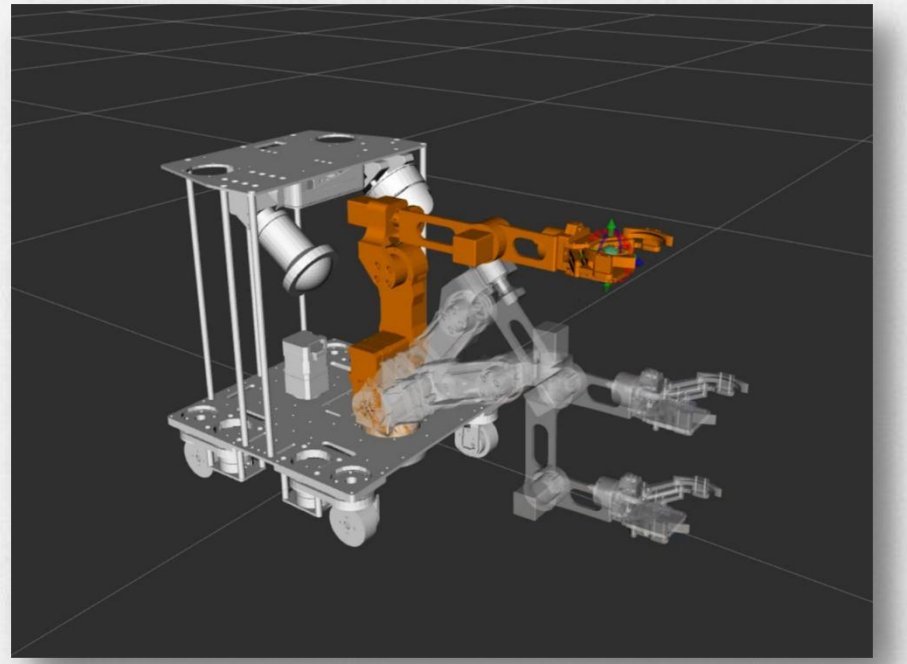
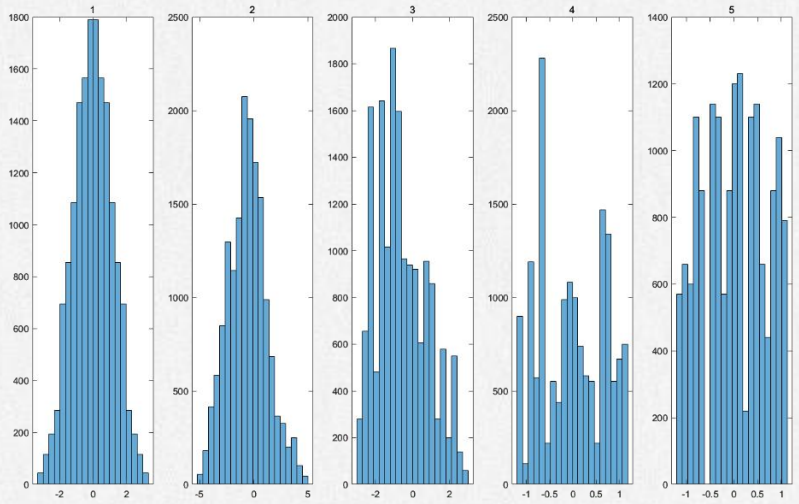


Figure 6: The dimensions of the Meca500 (R3 and R4)



MoveIt!



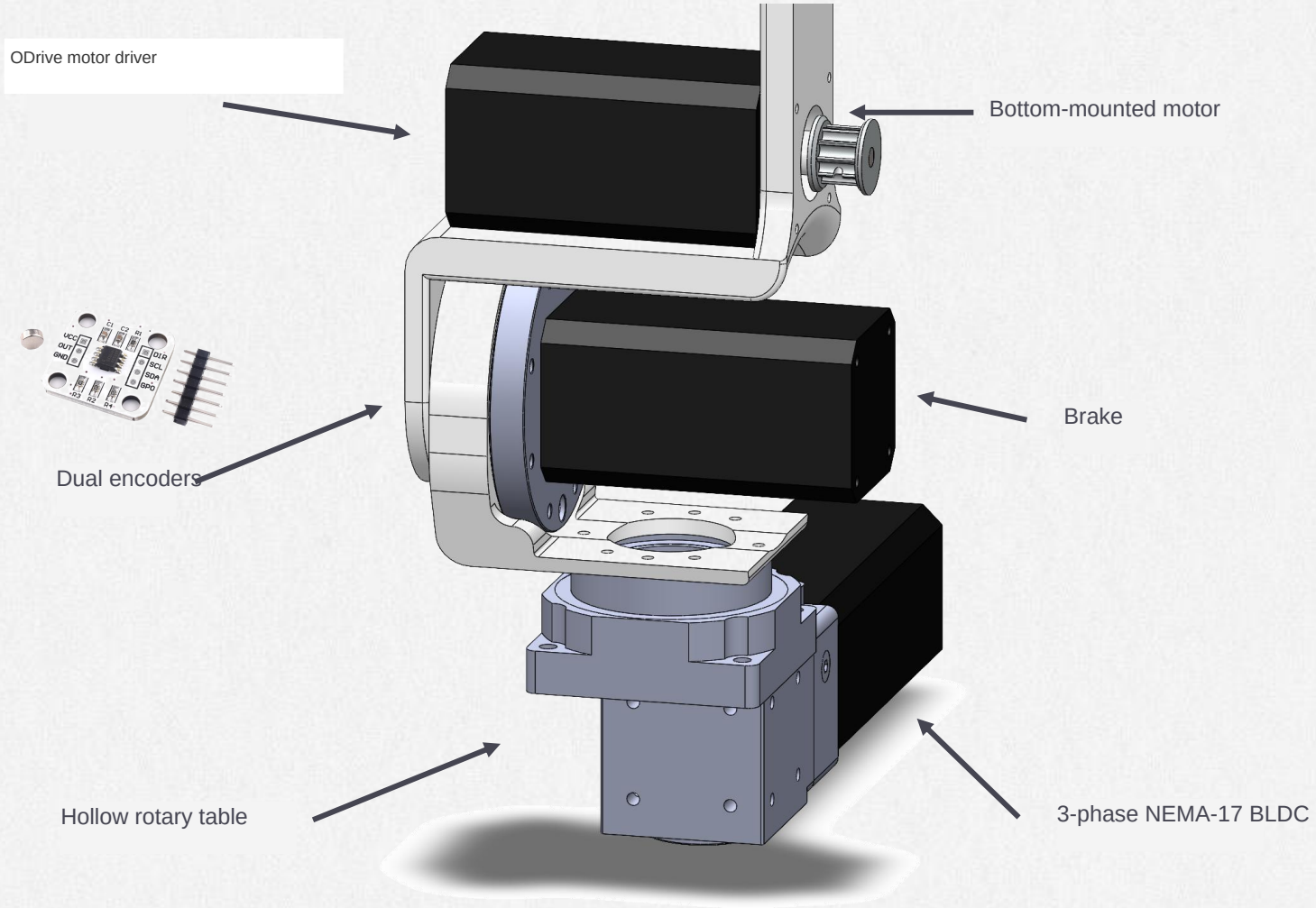
θ_i	d_i/mm	α_i	a_i/mm
θ_1	0	0	0
θ_2	0	$\frac{\pi}{2}$	0
θ_3	0	π	164
θ_4	120	$-\frac{\pi}{2}$	50
θ_5	0	$-\frac{\pi}{2}$	0

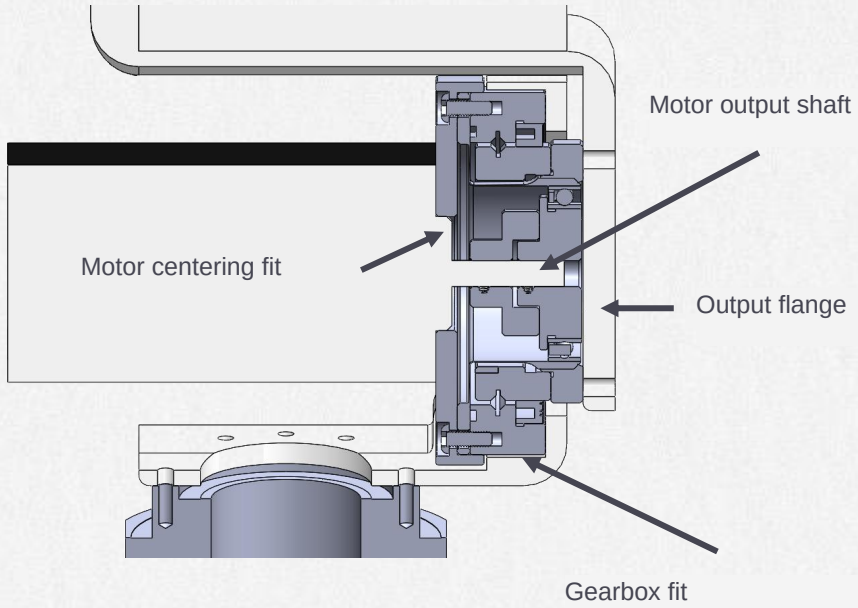
[1] Pieper D L. The kinematics of manipulators under computer control[M]. Stanford University, 1969.

Manipulator Design

A high-performance, reliable desktop 6-DoF arm

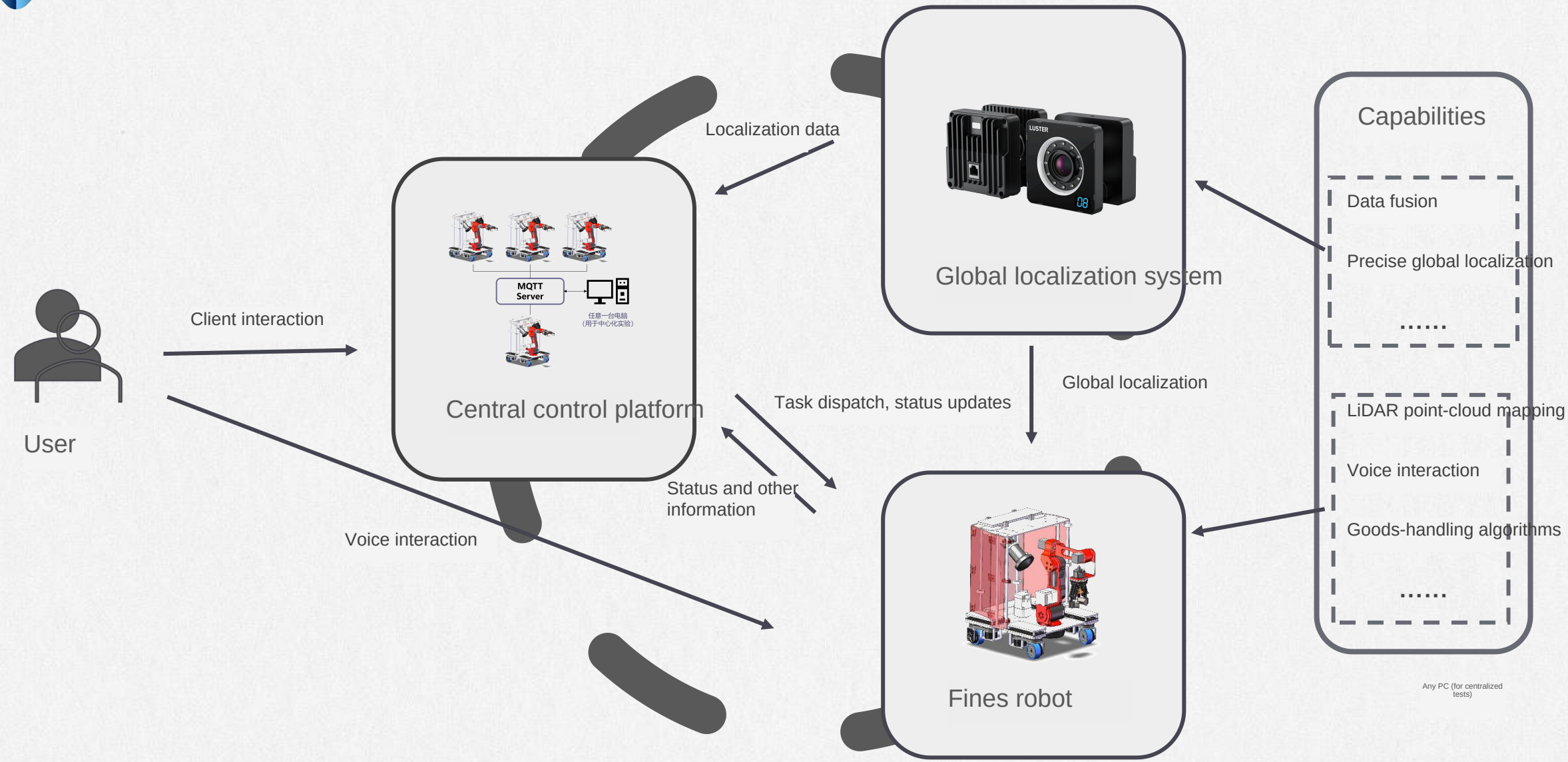
- Mechanics
- Control & sensing
- Performance & reliability





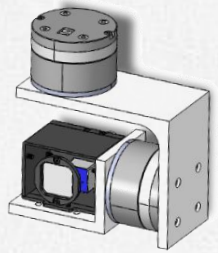
FineSched

Robot Scheduling System



Robot Testbed

Where all the key technologies come together

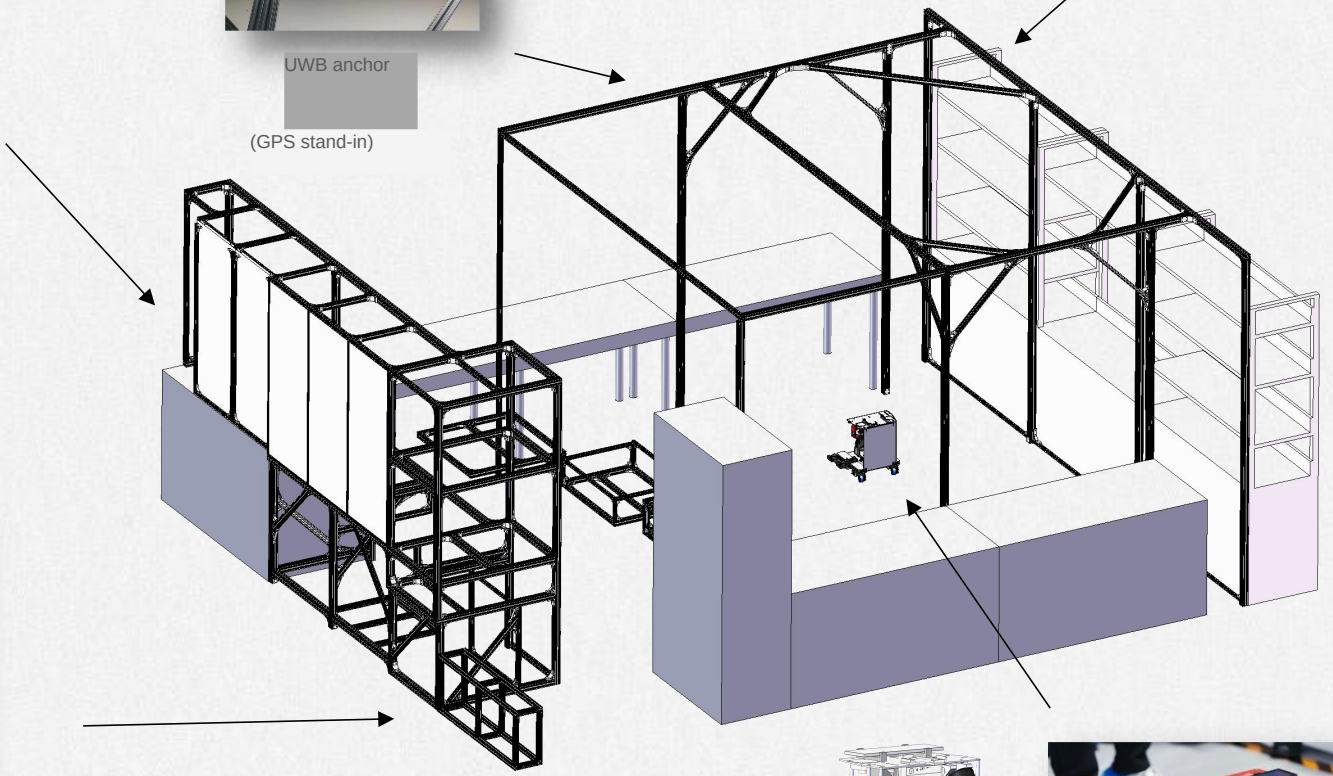


Optical tracking anchor

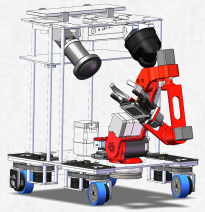
UWB anchor

(GPS stand-in)

Storage area



Display area

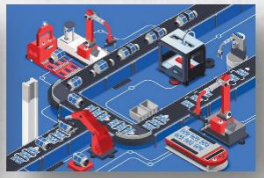


Individual robots

A multi-robot sandbox

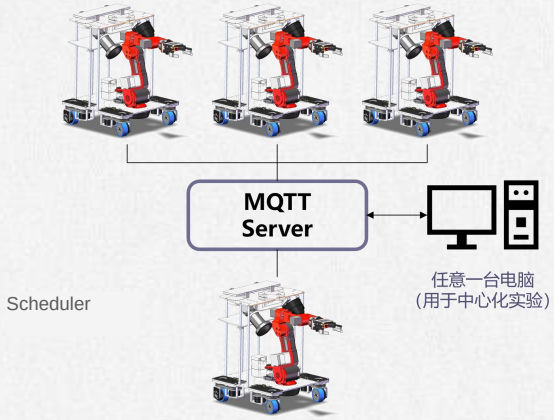
Simulating many application scenarios in the lab

Multi-agent attack/defense



Industry 4.0

Harsh-environment exploration



THANK YOU

Connecting Intelligence, Empowering the Future